Normalization of the Event Execution and Post-Event Analytics Platform

The ER diagram for the Event Execution and Post-Event Analytics Platform was normalized step-by-step to eliminate redundancy and maintain data integrity. The normalization process was applied from First Normal Form (1NF) to Boyce-Codd Normal Form (BCNF).

The following steps describe how the ER diagram was normalized from 1NF to BCNF.

❖ 1NF (First Normal Form) :

To achieve 1NF, all attributes were ensured to contain atomic values, and repeating groups were removed from entities in the ER diagram.

The following transformations were applied:

- Separated session information from Event

  Instead of storing multiple session topics within a single event record, a new entity **SESSION** was created.

  Before normalization:
  EVENT(event_id, title, session_topics)

  After applying 1NF:
  EVENT(event_id, title, description)
  SESSION(session_id, topic, start_time, end_time, event_id)

- Separated poll answers submitted by attendees

  Instead of storing multiple answers in a poll entity, a separate **POLL_RESPONSE** entity was introduced.

  Before normalization:
  POLL(poll_id, question, responses)

  After applying 1NF:
  POLL(poll_id, question, created_at)
  POLL_RESPONSE(response_id, poll_id, attendee_id, answer)

These changes ensured that each attribute stores a single value and no repeating groups exist, satisfying 1NF

❖ **2NF (Second Normal Form)** :

After achieving 1NF, the schema was further normalized to 2NF by eliminating partial dependencies on composite keys.

The following transformations were applied:

- Created RSVP relation for attendee participation

  The relationship between **ATTENDEE and EVENT** is many-to-many. Instead of storing multiple attendee details inside the event entity, the **RSVP** table was created.

  RSVP(event_id, attendee_id, rsvp_date, status)

  Here:
  (event_id, attendee_id) → rsvp_date, status

  This ensures attributes depend on the entire composite key.

- Separated poll answers into POLL_RESPONSE

  Poll answers depend on both the poll and the attendee.

  POLL_RESPONSE(poll_id, attendee_id, answer)

  Functional dependency:
  (poll_id, attendee_id) → answer

  This removes partial dependency.

❖ 3NF (Third Normal Form) :

To achieve 3NF, transitive dependencies were removed by separating attributes that depend on other non-key attributes.

The following changes were applied:

- Separated user roles from USER

  Instead of storing role-specific attributes inside the USER table, specialized entities were created:
  USER,CLIENT,EVENT_MANAGER,COORDINATOR,ATTENDEE

  This avoids storing unnecessary role attributes inside USER.

- Separated event analytics into REPORT

  Instead of storing analytics attributes inside EVENT, a new relation was introduced.

  EVENT(event_id, title, date, location)
  REPORT(report_id, event_id, attendance_count, poll_participation, feedback_score)

  This removes transitive dependency:
  event_id → report attributes

❖ BCNF (Boyce-Codd Normal Form) :

Finally, the schema was verified to satisfy BCNF, which requires that every determinant must be a candidate key.

The following conditions were verified:

- In **USER**, the primary key determines all attributes:

  user_id → name, email, phone, password

- In **EVENT**, the primary key determines all event attributes:

  event_id → title, description, date, location

- In **SESSION**, the session ID determines session attributes:

  session_id → topic, start_time, end_time

- In **POLL**, the poll ID determines the question and creation time:

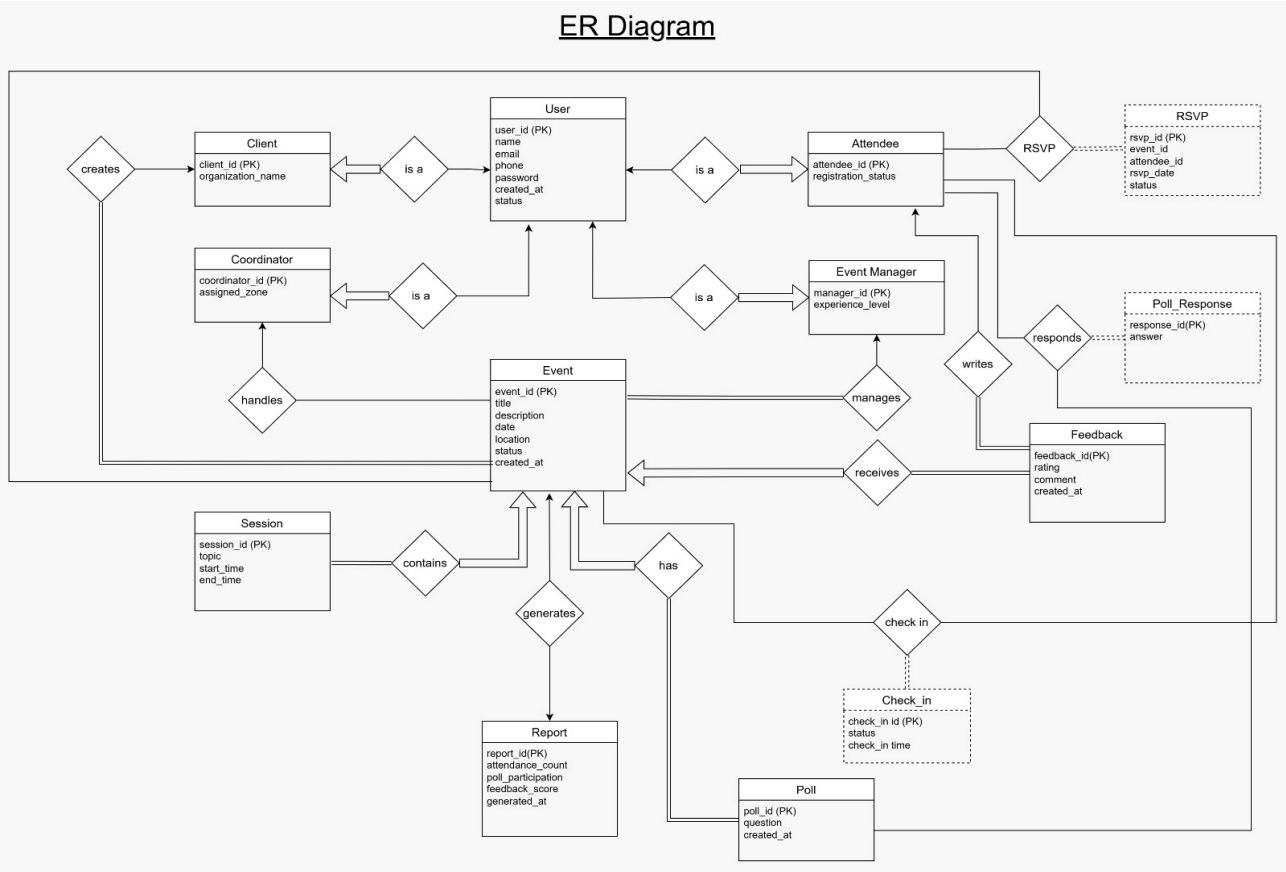
  poll_id → question, created_at

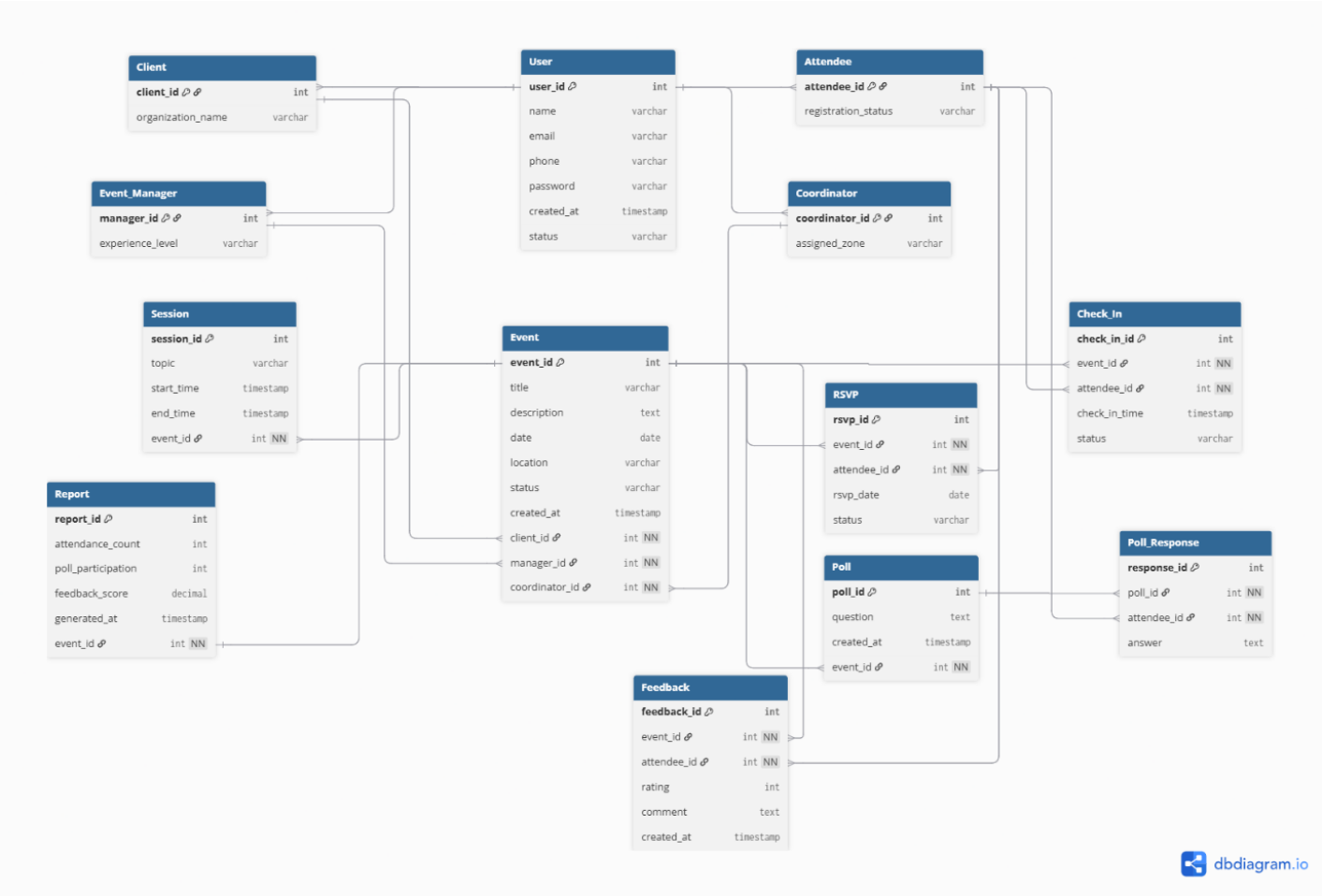
- In **REPORT**, the report ID determines analytics attributes:

  report_id → attendance_count, poll_participation, feedback_score

Since all relations satisfy the condition that every determinant is a candidate key, the schema satisfies BCNF.

❖ Therefore, all relations in the Event Execution and Post-Event Analytics Platform are normalized up to BCNF, ensuring that every determinant is a candidate key and eliminating redundancy and update anomalies.




Client
client_id int
organization_name varchar
User
user_id int
name varchar
email varchar
phone varchar
password varchar
created_at timestamp
status varchar
Attendee
attendee_id int
registration_status varchar
Event_Manager
manager_id int
experience_level varchar
Coordinator
coordinator_id int
assigned_zone varchar
Session
session_id int
topic varchar
start_time timestamp
end_time timestamp
event_id int NN
Event
event_id int
title varchar
description text
date date
location varchar
status varchar
created_at timestamp
client_id int NN
manager_id int NN
coordinator_id int NN
Check_In
check_in_id int
event_id int NN
attendee_id int NN
check_in_time timestamp
status varchar
RSVP
rsvp_id int
event_id int NN
attendee_id int NN
rsvp_date date
status varchar
Report
report_id int
attendance_count int
poll_participation int
feedback_score decimal
generated_at timestamp
event_id int NN
Poll
poll_id int
question text
created_at timestamp
event_id int NN
Poll_Response
response_id int
poll_id int NN
attendee_id int NN
answer text
Feedback
feedback_id int
event_id int NN
attendee_id int NN
rating int
comment text
created_at timestamp
dbdiagram.io